

Notițe - curs 4

[Curs 4](#)

Interfețe funcționale

- orice interfață de conține doar o **metodă abstractă**
- astfel putem omite numele metodei atunci când implementăm interfața și putem elimina folosirea claselor anonime -> în locul lor vom avea lambda expresii sau referințe la metode
- o interfață funcțională este adnotată cu `@FunctionalInterface`
- **Interfața funcțională** = o schemă pentru o funcție, cu o singură metodă.
- **Lambda** = o metodă super-simplă de a scrie funcții fără cod inutil.
- **Împreună**, poți crea și folosi funcții direct, fără a defini clase suplimentare.

Utilizarea interfețelor funcționale

ArithmeticExpression f = referintaLaOMetoda sau oExpresie sau oFunctieLambda

Ce este o interfață funcțională?

O **interfață funcțională** este ca o **schemă pentru o funcție** — o interfață care **are o singură metodă abstractă**. De exemplu, dacă vrei o interfață care să descrie o funcție simplă de calcul, cum ar fi adunarea, scăderea sau înmulțirea a două numere, atunci acea interfață va avea o singură metodă abstractă.

De ce e „funcțională”?

E „funcțională” pentru că e special creată să poată fi **folosită ca o funcție** în Java. Și, datorită unei reguli Java 8, dacă ai o singură metodă abstractă, poți scrie această funcție direct, fără cod în plus.

```
@FunctionalInterface
interface Calculator {
    double operate(double a, double b); // O singură metodă abstractă
}
```

Funcție lambda

O **expresie lambda** este o metodă foarte simplă de a scrie o funcție, fără să fie nevoie de mult cod. Practic, o lambda este o **funcție scrisă pe scurt**, folosită pentru a implementa rapid interfețele funcționale.

O funcție lambda (funcție anonimă) este o funcție definită și apelată fără a fi legată de un identificator.

De ce e utilă?

Înainte de lambdas, trebuia să creezi o **clasă separată** sau **clase anonime** pentru fiecare funcție simplă (ceea ce complica lucrurile). Cu lambda, poți scrie funcții pe loc, **fără să creezi clase noi**.

Cum arată o lambda?

Se scrie ca o **săgeată** -> între argumentele funcției și codul funcției.

Interfețe funcționale predefinite

Java 8 introduce interfețe funcționale predefinite, care sunt ca niște „instrumente” specifice pentru a face diverse lucruri cu datele. Gândește-te la ele ca la roluri pe care le poți atribui unor obiecte.

1. **Predicate** (funcție care primește un argument și returnează un boolean):

- Gândește-te la Predicate ca la un **filtru de acces**. Imaginează-ți că ești un portar care permite intrarea doar anumitor oameni într-o clădire. De exemplu, verifici dacă cineva are invitație (`true` dacă da, `false` dacă nu)

```
Predicate<String> esteMare = s -> s.length() > 5;
// false, deoarece „Salut” are 5 litere`
System.out.println(esteMare.test("Salut"));
```

2. **Consumer** (funcție care primește un argument și realizează o acțiune fără să returneze nimic):

- Imaginează-ți un **aparat de cafea** care ia boabele de cafea (argumentul) și le transformă în cafea fără să returneze boabele.

```
Consumer<String> printa = s -> System.out.println(s);
printa.accept("Salut lume!"); // Afișează „Salut lume!”
```

3. **Function** și **UnaryOperator** (funcții care primesc un argument și returnează un rezultat):

- Gândește-te la Function ca la un **traducător**. Primește un cuvânt într-o limbă și îl returnează tradus în altă limbă.
- UnaryOperator e ca un „Function” special, unde atât intrarea, cât și rezultatul sunt de același tip.

```
Function<String, Integer> lungimeaTextului = s -> s.length();
System.out.println(lungimeaTextului.apply("Salut")); // Va afișa 5
UnaryOperator<Integer> dublu = x -> x * 2;
System.out.println(dublu.apply(3)); // Va afișa 6
```

4. **Supplier** (funcție care nu primește argumente, dar returnează un rezultat):

- Imaginează-ți un **automat de mâncare**: apeși un buton și îți oferă un snack (rezultatul), fără să-i dai nimic în schimb.

```
Supplier<String> furnizorDeText = () -> "Salut din Supplier!";
System.out.println(furnizorDeText.get());
// Afișează „Salut din Supplier!”
...

```

5. **Comparator** (funcție care compară două obiecte):

- Gândește-te la **Comparator** ca la un **juriu de concurs** care compară participanții și decide care e mai **bun**. **Comparatorul** îți spune dacă un obiect e mai mare, mai mic sau egal cu altul.

```
``java
    Comparator<String> comparaLungime = (s1, s2) -> Integer.compare(s1.length(), s2.length());
    System.out.println(comparaLungime.compare("Salut", "Hello"));
    // 0, deoarece au aceeași lungime

```

Clasa Optional

- **Optional** e ca o **cutie sigură** pentru valori. Uneori o valoare poate să nu existe, iar fără acest „ambalaj”, ar putea apărea o eroare. De exemplu, în loc să riști o eroare când deschizi un sertar și nu găsești nimic înăuntru, te asiguri că sertarul poate fi fie gol, fie să conțină ceva. Astfel eviți erorile neașteptate.

```
Optional<String> optionalText = Optional.ofNullable(null);
// 0 cutie „goală” momentan
```

```
System.out.println(optionalText.orElse("Text implicit"));
// Afișează „Text implicit”
```

Metoda `ofNullable` din clasa `Optional` este o modalitate de a crea un obiect `Optional` care poate conține fie o valoare, fie `null`. Este o metodă de "siguranță" pentru a evita erorile `NullPointerException`.

Cum funcționează `Optional.ofNullable` ?

- Dacă valoarea pe care i-o dai lui `ofNullable` **nu este** `null`, `Optional` va conține acea valoare.
- Dacă valoarea **este** `null`, `Optional` va fi „gol” (adică, fără valoare).

De ce este utilă `ofNullable` ?

Scopul principal al `ofNullable` este să **evite folosirea directă a valorilor** `null`, oferind în schimb metode sigure de verificare a prezenței unei valori. În loc să folosești `if (x != null)`, poți să folosești `Optional` cu metodele lui specifice.

To add:

`forEach` - a apărut din nevoia de a parcurge indicii pentru web si astfel parcurge indicii respectivi deoarece o comunitate e mereu ordonata

@lambda python